

LAPORAN TUGAS BESAR STRATEGI ALGORITMA (IF2211)
Pemanfaatan Algoritma Greedy dalam Pembuatan Bot Permainan
Robocode Tank Royale



Disusun Oleh:

David Elbai Novialdi Maringan (124140058)

Duwi Putri Wahyuni (124140049)

Andini (124140192)

Dosen Pengampu: Winda Yulita, [M.Cs.](#)

TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI SUMATERA
2026

KATA PENGANTAR

Puji syukur kami panjatkan kepada Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan Tugas Besar mata kuliah IF2211 Strategi Algoritma ini dapat diselesaikan dengan baik. Laporan ini disusun untuk memenuhi komponen penilaian akademis dalam mengeksplorasi serta mengimplementasikan variasi algoritma Greedy pada platform simulasi Robocode Tank Royale.

Dalam proyek ini, kami merancang kecerdasan buatan berbentuk *bot* tangki otomatis yang mampu mengambil keputusan taktis secara *real-time* tanpa intervensi manusia. Kami mengucapkan terima kasih kepada Ibu Winda Yulita, M.Cs. selaku dosen pengampu yang telah memberikan arahan teknis. Kami menyadari laporan ini masih memiliki ruang perbaikan, oleh karena itu kritik konstruktif sangat kami harapkan.

DAFTAR ISI

KATA PENGANTAR.....	2
BAB I: DESKRIPSI TUGAS.....	6
1.1 Latar Belakang Masalah.....	6
1.2 Spesifikasi dan Target Masalah.....	6
BAB II: LANDASAN TEORI.....	7
2.1 Algoritma Greedy.....	7
2.1.1 Pengertian Algoritma Greedy.....	7
2.1.2 Komponen Algoritma Greedy.....	7
2.1.3 Kelebihan dan Kekurangan Algoritma Greedy.....	7
2.2 Robocode Tank Royale.....	7
2.2.1 Karakteristik Permainan.....	7
2.2.2 Implementasi Algoritma Greedy pada Bot.....	8
BAB III: APLIKASI STRATEGI GREEDY.....	9
3.1 Mapping Persoalan Robocode ke Elemen Greedy.....	9
3.2 Eksplorasi 4 Alternatif Strategi Berbasis Heuristic Berbeda.....	9
3.2.1 Alternatif 1: Aggressive Pursuit & Dynamic Firepower - “Ngaboot”.....	9
3.2.2 Alternatif 2: Weakest Target Ram + Bullet Combo- “RammerBot”.....	10
3.2.3 Alternatif 3: Expected Gain Maximization + Linear Targeting - “SniperBot”.....	10
3.2.4 Alternatif 4: Maximum Survival Score - “SurvivorBot”.....	10
3.3 Analisis Efisiensi dan Efektivitas Secara Teoritis.....	11
3.4 Desain Solusi Greedy Terpilih (Hybrid Dinamis Vektor Status).....	11
BAB IV: IMPLEMENTASI DAN PENGUJIAN.....	12
4.1 Representasi Logika Algoritma dalam Pseudocode.....	12
4.1.1 Pseudocode Bot Utama (Ngaboot).....	12
4.1.2 Pseudocode Bot Alternatif 2 (RammerBot).....	13
4.1.3 Pseudocode Bot Alternatif 3 (SniperBot).....	15
4.1.4 Pseudocode Bot Alternatif 4 (SurvivorBot).....	18
4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur Bot.....	22
4.2.1 Struktur Data.....	22
4.2.2 Fungsi-Fungsi Utama.....	22
4.2.3 Prosedur Event Handler.....	23
4.3 Pengujian.....	23
4.3.1 Pengujian 1:.....	23
4.3.2 Pengujian 2:.....	24
4.3.3 Pengujian 3:.....	25
4.3.4 Pengujian 4:.....	26
4.4 Analisis Hasil Pengujian.....	27
BAB V: KESIMPULAN DAN SARAN.....	28

5.1 Kesimpulan.....	28
5.2 Saran.....	28
LAMPIRAN.....	29
LAMPIRAN A : Tautan Repository Github.....	29
LAMPIRAN B ; Tautan Video Demo.....	29
LAMPIRAN C : Tabel Pemenuhan Tugas.....	29
DAFTAR PUSTAKA.....	30

DAFTAR TABEL

Tabel 1. Analisis Pengujian satu.....	24
Tabel 2. Analisis Pengujian dua.....	24
Tabel 3. Analisis Pengujian satu.....	25
Tabel 4. Analisis Pengujian satu.....	26

DAFTAR GAMBAR

Gambar 1. Pengujian satu Ngaboot vs RammerBot.....	24
Gambar 2. Pengujian dua Ngaboot vs SniperBot.....	25
Gambar 3. Pengujian satu Ngaboot vs SurvivorBot.....	26
Gambar 4. Pengujian satu Ngaboot vs TrackFireBot.....	27

BAB I: DESKRIPSI TUGAS

1.1 Latar Belakang Masalah

Robocode Tank Royale merupakan lingkungan simulasi pertempuran di mana beberapa *bot* tangki virtual saling bertarung di dalam arena dua dimensi hingga hanya tersisa satu pemenang utama (*Battle Royale*). Tantangan terbesar dalam platform ini adalah sifat lingkungan pertempuran yang sangat dinamis, tidak pasti (*uncertainty*), serta berjalan secara waktu nyata (*real-time*). Setiap *bot* dituntut untuk melakukan pemindaian radar, menentukan arah gerak roda, mengarahkan laras meriam, dan mengeksekusi tembakan dalam hitungan milidetik.

1.2 Spesifikasi dan Target Masalah

Masalah utama yang diselesaikan dalam tugas besar ini adalah bagaimana membangun logika penalaran (*decision-making*) pada *bot* agar mampu memenangkan kompetisi dengan mengumpulkan poin kerusakan (*damage score*) tertinggi sekaligus mempertahankan energi hidup (*survival score*) semaksimal mungkin. Pendekatan konvensional yang mengeksplorasi seluruh ruang kemungkinan langkah (seperti *Brute Force* atau *Backtracking*) tidak layak digunakan karena memiliki kompleksitas waktu yang tinggi, sehingga akan menyebabkan *bot* terlambat merespons serangan lawan (terjadi *lag*). Oleh karena itu, diperlukan strategi berbasis Algoritma Greedy untuk menghasilkan keputusan instan yang cepat dan efisien.

BAB II: LANDASAN TEORI

2.1 Algoritma Greedy

2.1.1 Pengertian Algoritma Greedy

Algoritma Greedy merupakan metode pemecahan masalah optimasi yang mencari solusi selangkah demi selangkah (*step-by-step*). Karakteristik utama dari algoritma ini adalah prinsip "*take what you can get now*", yaitu mengambil pilihan yang terlihat paling menguntungkan pada saat itu (lokal optimum) tanpa mempertimbangkan dampak dari keputusan tersebut di masa depan. Diperlukan sebuah pembuktian atau pengujian empiris untuk melihat apakah rangkaian solusi lokal optimum tersebut berhasil membentuk solusi global optimum atau justru terjebak pada kondisi suboptimal.

2.1.2 Komponen Algoritma Greedy

Secara formal, Algoritma Greedy tersusun atas lima komponen utama:

1. Himpunan Kandidat (C): Elemen-elemen yang akan dipilih untuk membentuk solusi.
2. Himpunan Solusi (S): Kandidat yang telah terpilih dan berhasil membentuk solusi akhir.
3. Fungsi Seleksi (Selection Function): Aturan heuristik yang menentukan kandidat mana yang paling optimal untuk dipilih pada setiap langkah.
4. Fungsi Kelayakan (Feasibility Check): Pengujian untuk memastikan kandidat yang dipilih tidak melanggar batasan (constraints) sistem.
5. Fungsi Objektif: Nilai maksimisasi atau minimasi yang ingin dicapai di akhir proses optimasi.

2.1.3 Kelebihan dan Kekurangan Algoritma Greedy

- **Kelebihan:** Memiliki efisiensi performa yang sangat tinggi dengan kompleksitas waktu minimal ($O(1)$ atau $O(n)$) karena tidak memerlukan proses pencarian mundur (*backtracking*). Sangat andal untuk sistem komputasi *real-time*.
- **Kekurangan:** Sifatnya pandang pendek (*myopic*) membuat algoritma ini rentan melewati solusi global terbaik jika pilihan langkah di awal mengorbankan peluang yang jauh lebih menguntungkan di langkah akhir.

2.2 Robocode Tank Royale

2.2.1 Karakteristik Permainan

Dalam Tank Royale, setiap robot dikonstruksikan atas tiga komponen mekanis terpisah yang arah rotasinya dapat berputar secara independen:

1. Body (Badan): Mengatur pergeseran translasi (maju/mundur) serta sudut kemudi pergerakan tank.
2. Gun (Meriam): Berfungsi membidikkan laras tembakan ke arah koordinat target secara presisi.
3. Radar: Berfungsi memancarkan gelombang pemindai untuk menangkap variabel data musuh (jarak, koordinat, dan sisa darah).

2.2.2 Implementasi Algoritma Greedy pada Bot

Program bot bekerja menggunakan mekanisme *event-driven*. Saat sapuan sensor radar mendeteksi objek lawan, sistem permainan akan memicu kejadian (*event*) **OnScannedBot**. Data mentah musuh tersebut langsung diolah oleh logika algoritma greedy untuk memerintahkan rotasi badan, pelacakan meriam, dan penembakan peluru secara simultan dalam satu *tick*.

BAB III: APLIKASI STRATEGI GREEDY

3.1 Mapping Persoalan Robocode ke Elemen Greedy

Untuk menerapkan paradigma Greedy ke dalam sistem kendali kecerdasan robot komponen permainan dipetakan secara matematis ke dalam 5 elemen berikut:

1. **Himpunan Kandidat (C):** Kumpulan semua instruksi aksi atomik fisik yang valid disediakan oleh API Robocode pada setiap *tick*, yaitu: {*Maju()*, *Mundur()*, *PutarBadan()*, *PutarGun()*, *PutarRadar()*, *Tembak(daya)*, *Diam()*}.
2. **Himpunan Solusi (S):** Rangkaian urutan kronologis aksi yang telah dipilih dan dieksekusi oleh *bot* dari awal ronde pertarungan hingga selesai.
3. **Fungsi Seleksi:** Logika pembobotan (*heuristic function*) yang mengevaluasi himpunan kandidat berdasarkan parameter masukan *real-time* (seperti Jarak Euclidean musuh dan Sisa Energi pribadi) untuk memilih satu aksi instan terbaik.
4. **Fungsi Kelayakan (Feasibility Check):** Pengujian kondisi untuk memastikan aksi terpilih tidak melanggar hukum batas permainan. Contoh: Aksi *Tembak(daya)* hanya layak jika energi internal > daya peluru; Aksi *Maju()* hanya layak jika posisi masa depan tidak menabrak tembok (*boundary check*).
5. **Fungsi Objektif:** Maksimisasi akumulasi skor akhir turnamen yang dihitung dari formula: $\text{Skor} = \text{Skor_Kerusakan} + \text{Skor_Eliminasi} + \text{Skor_Survival}$.

3.2 Eksplorasi 4 Alternatif Strategi Berbasis Heuristic Berbeda

3.2.1 Alternatif 1: Aggressive Pursuit & Dynamic Firepower - “Ngaboot”

Pada alternatif utama menggunakan strategi Heuristik Agresif yang berfokus pada penyerangan langsung terhadap musuh dengan keputusan sederhana berbasis jarak dan energi. Bot akan terus mencari musuh, mendekati target ketika jauh, lalu menyerang dengan kekuatan peluru yang menyesuaikan sisa energi

1. Heuristik yang digunakan:
 - Jika energi tinggi → gunakan firepower besar.
 - Jika musuh jauh → dekati target.
 - Jika musuh dekat → bergerak menghindar sambil menyerang.
2. Kelebihan:
 - Serangan konsisten.
 - Cocok melawan bot pasif.
3. Kekurangan:
 - Kurang efektif terhadap bot dengan pergerakan tidak terduga.
 - Penghindaran masih sederhana.

3.2.2 Alternatif 2: Weakest Target Ram + Bullet Combo- “RammerBot”

Pada strategi lternatif kedua menggunakan heuristic pemilihan target berdasarkan energi terendah. Bot akan menyimpan data semua musuh, lalu memilih lawan paling lemah untuk dikejar dan ditabrak.

1. Heuristik yang digunakan:
 - Pilih musuh dengan energi paling kecil.
 - Kejar target hingga jarak dekat.
 - Gunakan tabrakan dan tembakan ringan untuk finishing.
2. Kelenihan:
 - Efektif mengeliminasi lawan satu per satu.
 - Mempercepat pengurangan jumlah musuh.
3. Kekurangan:
 - Rentan saat mengejar target.
 - Kurang aman jika banyak musuh aktif.

3.2.3 Alternatif 3: Expected Gain Maximization + Linear Targeting - “SniperBot”

Strategi ini memakai pendekatan prediksi gerakan musuh dan evaluasi efektivitas peluru. Bot menghitung peluang serangan berdasarkan jarak, kecepatan peluru, dan estimasi posisi lawan.

1. Heuristik yang digunakan:
 - Pilih firepower dengan keuntungan terbesar.
 - Prediksi posisi musuh sebelum menembak.
 - Jaga jarak optimal dari lawan.
2. Kelebihan:
 - Akurasi tembakan lebih tinggi.
 - Penggunaan energi lebih efisien.
3. Kekurangan:
 - Perhitungan lebih kompleks.
 - Kurang efektif pada pertarungan jarak dekat.

3.2.4 Alternatif 4: Maximum Survival Score - “SurvivorBot”

Strategi ini fokus pada bertahan hidup dengan mendeteksi kemungkinan serangan lawan dan melakukan penghindaran acak.

1. Heuristik yang digunakan:
 - Deteksi musuh menembak dari penurunan energi.
 - Bergerak acak untuk menghindari peluru.
 - Menjaga jarak aman dari lawan.
2. Kelebihan:
 - Sulit terkena peluru.
 - Cocok untuk pertahanan jangka panjang.
3. Kekurangan:

- Damage output lebih kecil.
- Membutuhkan waktu lebih lama untuk menang.

3.3 Analisis Efisiensi dan Efektivitas Secara Teoritis

Berdasarkan analisis teoritis, Strategi Aggressive Pursuit & Dynamic Firepower sangat efektif menghadapi bot dengan pergerakan sederhana atau pasif, namun kurang efisien dalam penggunaan energi ketika banyak tembakan meleset. Weakest Target Ram + Bullet Combo memiliki kemampuan eliminasi cepat terhadap target lemah, tetapi berisiko mengurangi energi bot akibat tabrakan berulang saat melakukan ramming. Expected Gain Maximization + Linear Targeting mampu meningkatkan akurasi tembakan dan efisiensi firepower, namun kurang efektif menghadapi musuh dengan pola gerakan cepat dan tidak terprediksi. Sementara itu, Maximum Survival Score sangat baik dalam mempertahankan hidup bot di arena, tetapi menghasilkan damage dan perolehan skor yang relatif lebih rendah dibanding strategi agresif.

- *Aggressive Pursuit & Dynamic Firepower* sangat efektif membunuh *bot* tipe stasioner, namun tidak efisien dalam manajemen energi jika peluru meleset.
- *Weakest Target Ram + Bullet Combo* memiliki efektivitas disrupti gerakan yang tinggi, namun tidak efisien karena merusak struktur energi internal sendiri akibat benturan beruntun.
- *Expected Gain Maximization + Linear Targeting* memiliki akurasi tembakan efisien, namun kurang efektif menghadapi musuh bertipe gerak cepat.
- *Maximum Survival Score* memiliki efektivitas ketahanan hidup yang tinggi, namun efisiensi pengumpulan poinnya buruk sehingga jarang memenangkan turnamen berbasis total skor.

3.4 Desain Solusi Greedy Terpilih (Hybrid Dinamis Vektor Status)

Berdasarkan analisis trade-off dari berbagai strategi yang telah diuji, solusi yang dipilih adalah pendekatan **State-Based Adaptive Greedy Strategy**. Strategi ini menerapkan pemilihan aksi secara greedy berdasarkan kondisi real-time permainan, khususnya jarak musuh dan sisa energi bot (*Eself*).

Pada saat energi tinggi ($E_{self} > 50$), bot memprioritaskan aksi agresif dengan penggunaan *firepower* besar untuk memaksimalkan *damage output* terhadap lawan. Ketika energi berada pada tingkat menengah, bot tetap mempertahankan pola serangan namun menggunakan kekuatan peluru yang lebih efisien agar konsumsi energi tetap terjaga. Sementara itu, saat energi rendah, bot mengurangi penggunaan *firepower* dan lebih banyak melakukan pergerakan menghindar untuk mempertahankan keberlangsungan hidup di arena.

BAB IV: IMPLEMENTASI DAN PENGUJIAN

4.1 Representasi Logika Algoritma dalam Pseudocode

4.1.1 Pseudocode Bot Utama (Ngaboot)

START

Deklarasi:

enemyX = -1

enemyY = -1

enemyDistance = tak hingga

Saat bot dijalankan:

Atur warna body = Cyan

Atur warna turret = Red

Atur warna radar = Pink

Atur warna peluru = Green

Selama bot masih berjalan:

Putar radar ke kanan 45 derajat

Jika musuh belum terdeteksi:

Maju 50

Belok kanan 30

Saat musuh terdeteksi:

Simpan posisi X musuh

Simpan posisi Y musuh

Hitung jarak musuh

Jika energi > 50:

firePower = 3.0

Jika energi > 20:

firePower = 1.5

Selain itu:

firePower = 0.5

Hitung arah musuh

Arahkan senjata ke musuh

Tembak dengan firePower

Jika jarak musuh > 200:

Hadap ke arah musuh

```

    Maju sejauh minimum(jarak musuh / 2, 100)
    Selain itu:
        Belok kanan 90
        Maju 50

    Saat terkena peluru:
        Hitung arah datang peluru
        Belok kiri sesuai arah peluru
        Maju 75

    Saat menabrak dinding:
        Mundur 30
        Belok kanan 45

    Saat menabrak bot lain:
        Maju 30

    END

```

4.1.2 Pseudocode Bot Alternatif 2 (RammerBot)

```

START

Deklarasi:
    targetId = -1
    targetX = -1
    targetY = -1
    targetEnergy = tak hingga

    knownEnemies = daftar musuh yang diketahui

    Saat bot dijalankan:
        Selama bot masih berjalan:

            Putar radar ke kanan 45 derajat

            Jika target tersedia:
                Panggil ChaseTarget()
            Jika tidak:
                Belok kanan 20
                Maju 80

PROSEDUR ChaseTarget()
    Hitung jarak ke target

```

Hitung arah target

Hadap ke arah target

Jika jarak target > 20:

Maju menuju target

Arahkan senjata ke target

Jika gunHeat = 0 DAN energi > 10:

Tembak dengan power 0.5

END PROSEDUR

Saat musuh terdeteksi:

Simpan data musuh:

id

posisi x

posisi y

energi

Panggil SelectWeakestTarget()

Jika musuh yang dipindai adalah target utama:

Arahkan radar ke target

PROSEDUR SelectWeakestTarget()

bestId = -1

lowestEnergy = tak hingga

Untuk setiap musuh dalam knownEnemies:

Jika energi musuh lebih kecil dari lowestEnergy:

lowestEnergy = energi musuh

bestId = id musuh

simpan posisi musuh

Jika ditemukan target:

targetId = bestId

targetX = posisi x target

```
targetY = posisi y target  
targetEnergy = lowestEnergy
```

END PROSEDUR

Saat menabrak bot:

```
Maju 50
```

```
Jika bot ada dalam knownEnemies:  
  Perbarui energi musuh
```

Saat bot musuh mati:

```
Hapus musuh dari knownEnemies
```

```
Jika musuh yang mati adalah target utama:  
  Reset targetId  
  Reset targetX  
  Reset targetY  
  Reset targetEnergy
```

```
Panggil SelectWeakestTarget()
```

Saat terkena peluru:

```
Jika energi < 20:  
  Hitung arah peluru  
  Belok kiri menghindar  
  Maju 60
```

Saat menabrak dinding:

```
Mundur 20  
Belok kanan 30
```

END

4.1.3 Pseudocode Bot Alternatif 3 (SniperBot)

START

Deklarasi:

```
enemyX = -1  
enemyY = -1  
enemyEnergy = 100  
scanMissCount = 0
```

```
OPTIMAL_MIN_DIST = 250  
OPTIMAL_MAX_DIST = 500
```

Saat bot dijalankan:

Selama bot masih berjalan:

```
scanMissCount = scanMissCount + 1
```

Jika scanMissCount > 5:

Putar radar kanan 45

Belok kanan 15

Maju 40

Jika tidak:

Putar radar kanan 20

Saat musuh terdeteksi:

```
scanMissCount = 0
```

Simpan posisi X musuh

Simpan posisi Y musuh

Hitung jarak musuh

Simpan energi musuh sebelumnya

Perbarui energi musuh

```
bestFP = 0.1
```

```
bestGain = nilai minimum
```

Daftar firepower:

```
[0.5, 1.0, 1.5, 2.0, 2.5, 3.0]
```

Untuk setiap firepower:

Jika energi bot <= firepower:

lanjut ke firepower berikutnya

Hitung kecepatan peluru

Hitung waktu peluru mencapai target

Hitung peluang kena target

Hitung gain:

$gain = (3 \times \text{firepower} \times \text{peluang kena}) - \text{firepower}$

Jika gain lebih besar dari bestGain:

bestGain = gain

bestFP = firepower

Hitung kembali:

kecepatan peluru terbaik

waktu menuju target

Prediksi posisi musuh:

predictedX

predictedY

Arahkan senjata ke posisi prediksi musuh

Jika bestGain > 0:

Tembak dengan bestFP

Jika jarak musuh < OPTIMAL_MIN_DIST:

Balik arah menjauhi musuh

Maju sesuai selisih jarak

Jika jarak musuh > OPTIMAL_MAX_DIST:

Hadap ke arah musuh

Maju mendekati musuh

Jika jarak berada di antara optimal:

Bergerak menyamping

Maju 60

Arahkan radar ke musuh

Saat terkena peluru:

Hitung arah datang peluru

Belok kiri untuk menghindar

Maju 100

Belok kanan 30

Saat menabrak dinding:

Mundur 40

Belok kanan 60

Saat menabrak bot lain:

Mundur 60

Belok kanan 45

END

4.1.4 Pseudocode Bot Alternatif 4 (SurvivorBot)

START

Deklarasi:

enemyX = -1

enemyY = -1

enemyEnergy = 100

isEvading = false

evadeTurns = 0

moveDirection = 1

DANGER_DISTANCE = 200

SAFE_DISTANCE = 400

Saat bot dijalankan:

Selama bot masih berjalan:

Jika isEvading = true DAN evadeTurns > 0:

evadeTurns = evadeTurns - 1

Bergerak sesuai moveDirection sejauh 30

Jika tidak:
isEvading = false
Panggil PerformSurvivalMovement()

Putar radar ke kanan 45

PROSEDUR PerformSurvivalMovement()

Jika posisi musuh diketahui:

Hitung jarak ke musuh

Jika jarak musuh < DANGER_DISTANCE:

Hitung arah menjauh dari musuh

Belok menjauh

Maju 80

Jika tidak:

Bergerak zig-zag

Belok kanan sesuai arah gerak

Maju 50

Jika musuh tidak diketahui:

Belok kanan 15

Maju 50

END PROSEDUR

Saat musuh terdeteksi:

Simpan energi musuh sebelumnya

Perbarui:

enemyX

enemyY

enemyEnergy

Hitung jarak musuh

Hitung penurunan energi musuh

Jika energi musuh turun antara 0 sampai 3:
Musuh dianggap baru menembak

isEvading = true
evadeTurns = 5

Hitung sudut menghindar
Belok ke arah menghindar

Tentukan arah gerak acak:
1 atau -1

Bergerak sesuai arah sejauh 100

Jika jarak musuh < 300 DAN energi bot > 30:

Jika jarak < 150:
firePower = 1.0
Jika tidak:
firePower = 0.5

Arahkan senjata ke musuh
Tembak dengan firePower

Jika jarak musuh < DANGER_DISTANCE:

isEvading = true
evadeTurns = 8

Belok menjauh dari musuh
Maju 120

FUNGSI RandomBoolean()

Menghasilkan nilai:
true atau false secara acak

END FUNGSI

Saat terkena peluru:

```
isEvading = true
evadeTurns = 6

Hitung arah datang peluru

Belok untuk menghindari

Jika arah peluru positif:
    moveDirection = 1
Jika tidak:
    moveDirection = -1

Bergerak sesuai arah sejauh 100

Saat menabrak dinding:

    Mundur 30
    Belok kanan 60
    isEvading = false

Saat menabrak bot lain:

    Mundur 50
    Belok kiri 45
    isEvading = false

Saat bot musuh mati:
    Tidak ada aksi

END
```

4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur Bot

4.2.1 Struktur Data

Pada implementasi bot Robocode, struktur data digunakan untuk menyimpan informasi penting yang dibutuhkan selama pertandingan berlangsung. Data yang disimpan meliputi posisi musuh, energi musuh, arah gerakan, status penghindaran, serta target utama. Beberapa bot seperti RammerBot menggunakan struktur data Dictionary untuk menyimpan data banyak musuh berdasarkan ID masing-masing, sehingga bot dapat memilih target dengan energi paling rendah. Selain itu, variabel bertipe double digunakan untuk menyimpan koordinat posisi dan jarak musuh, sedangkan variabel bertipe boolean digunakan untuk menentukan kondisi tertentu seperti status evasive movement pada SurvivorBot.

4.2.2 Fungsi-Fungsi Utama

- **Run()**
Fungsi utama yang dijalankan terus-menerus selama pertandingan berlangsung. Fungsi ini mengatur pergerakan dasar bot, pencarian musuh menggunakan radar, serta pemanggilan strategi utama masing-masing bot.
- **ChaseTarget()**
Digunakan pada RammerBot untuk mengejar target yang telah dipilih. Fungsi ini mengatur arah gerakan menuju musuh, mendekati target, serta melakukan penembakan ketika target berada dalam jangkauan.
- **PerformSurvivalMovement()**
Digunakan pada SurvivorBot untuk mengatur pergerakan bertahan hidup. Fungsi ini membuat bot menjaga jarak aman dari musuh dan melakukan gerakan menghindar ketika lawan terlalu dekat.
- **SelectWeakestTarget()**
Digunakan pada RammerBot untuk memilih target dengan energi terendah dari seluruh musuh yang terdeteksi.
- **RandomBoolean()**
Fungsi pembantu pada SurvivorBot yang digunakan untuk menghasilkan nilai acak true atau false guna menentukan arah gerakan penghindaran.
- **DistanceTo()**
Fungsi bawaan Robocode yang digunakan untuk menghitung jarak antara bot dengan posisi musuh.
- **BearingTo()**
Fungsi untuk menghitung arah atau sudut posisi musuh terhadap bot.
- **GunBearingTo()**
Fungsi untuk menentukan arah putaran senjata menuju target agar peluru lebih akurat.

- **CalcBearing()**

Fungsi untuk menghitung arah datangnya peluru atau objek tertentu sehingga bot dapat melakukan penghindaran.

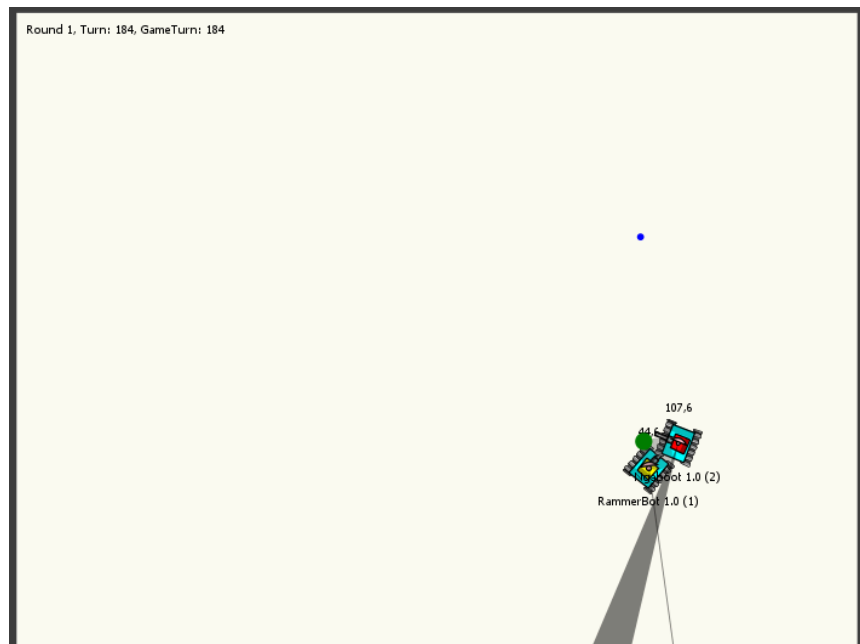
4.2.3 Prosedur Event Handler

Event handler merupakan prosedur yang dijalankan secara otomatis ketika suatu kondisi tertentu terjadi selama pertandingan. Prosedur OnScannedBot() digunakan ketika radar mendeteksi musuh dan biasanya berisi logika penargetan, perhitungan jarak, serta proses penembakan. Prosedur OnHitByBullet() digunakan untuk merespons ketika bot terkena peluru dengan melakukan evasive movement atau perubahan arah. Selanjutnya, OnHitWall() dijalankan ketika bot menabrak dinding sehingga bot dapat mundur dan mengubah arah gerakan. Prosedur OnHitBot() digunakan saat terjadi tabrakan antar bot untuk mengatur respons seperti menyerang atau menjauh. Beberapa bot juga menggunakan OnBotDeath() untuk memperbarui data target ketika salah satu musuh berhasil dikalahkan.

4.3 Pengujian

4.3.1 Pengujian 1: Ngaboot (Main Bot) VS RammerBot

Pengujian ini dilakukan untuk membandingkan strategi agresif seimbang milik Ngaboot dengan strategi kinetic ramming dari RammerBot. Pada pengujian ini diamati kemampuan Ngaboot dalam menjaga jarak dan menyerang dibandingkan kemampuan RammerBot dalam mengejar serta menabrak target secara langsung.



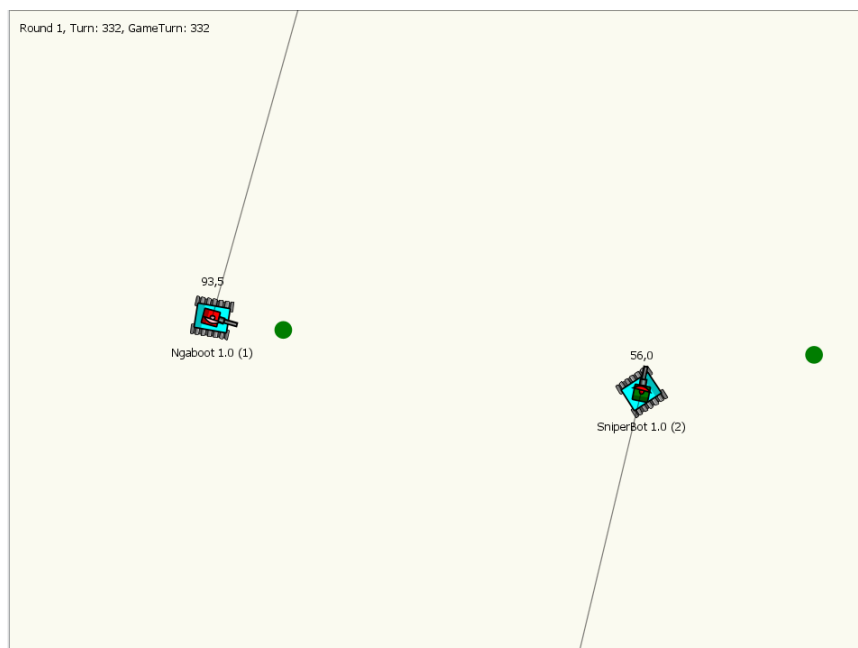
Gambar 1. Pengujian satu Ngaboot vs RammerBot

Tabel 1. Analisis Pengujian satu

Parameter	Bot Utama	Alternatif Bot
Skor Akhir	2271	846
Energi Tersisa	500	0
Jumlah Tembakan	624	42
Akurasi Tembakan	83	0
Hasil Pertandingan	10/0/0	0/10/0

4.3.2 Pengujian 2:Ngaboot (Main Bot) VS SniperBot

Pengujian ini bertujuan membandingkan strategi serangan langsung milik Ngaboot dengan strategi predictive shooting yang digunakan SniperBot. Fokus pengujian berada pada akurasi tembakan, pengaturan jarak, serta efektivitas pengambilan keputusan saat menyerang lawan.



Gambar 2. Pengujian dua Ngaboot vs SniperBot

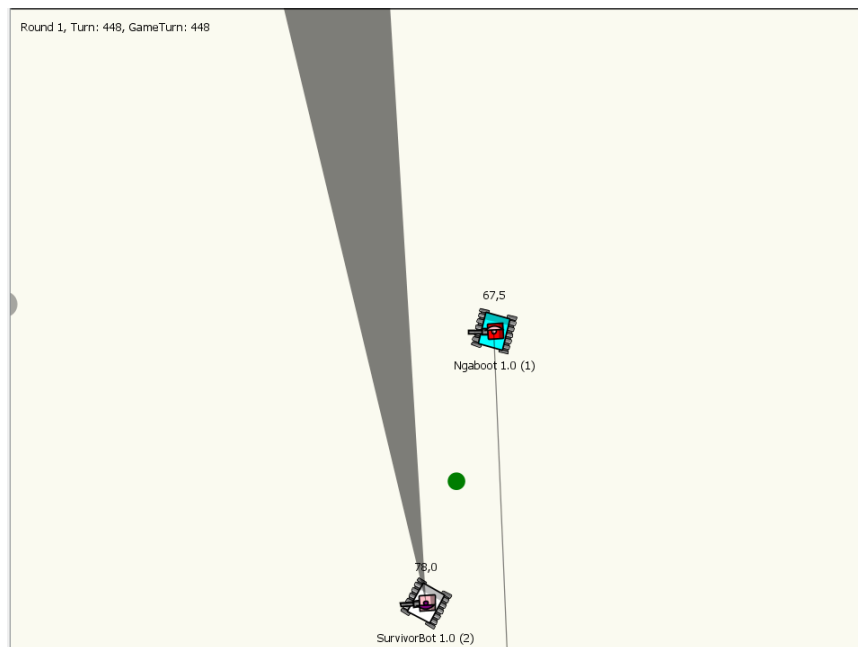
Tabel 2. Analisis Pengujian dua

Parameter	Bot Utama	Alternatif Bot
Skor Akhir	1482	130
Energi Tersisa	400	0

Parameter	Bot Utama	Alternatif Bot
Jumlah Tembakan	864	130
Akurasi Tembakan	137	0
Hasil Pertandingan	9/0/0	0/9/0

4.3.3 Pengujian 3:Ngaboot (Main Bot) VS SurvivorBot

Pengujian ini dilakukan untuk melihat performa bot agresif melawan bot bertahan hidup. Ngaboot berusaha memberikan tekanan melalui serangan aktif, sedangkan SurvivorBot mengutamakan evasive movement dan menjaga jarak aman untuk bertahan lebih lama.



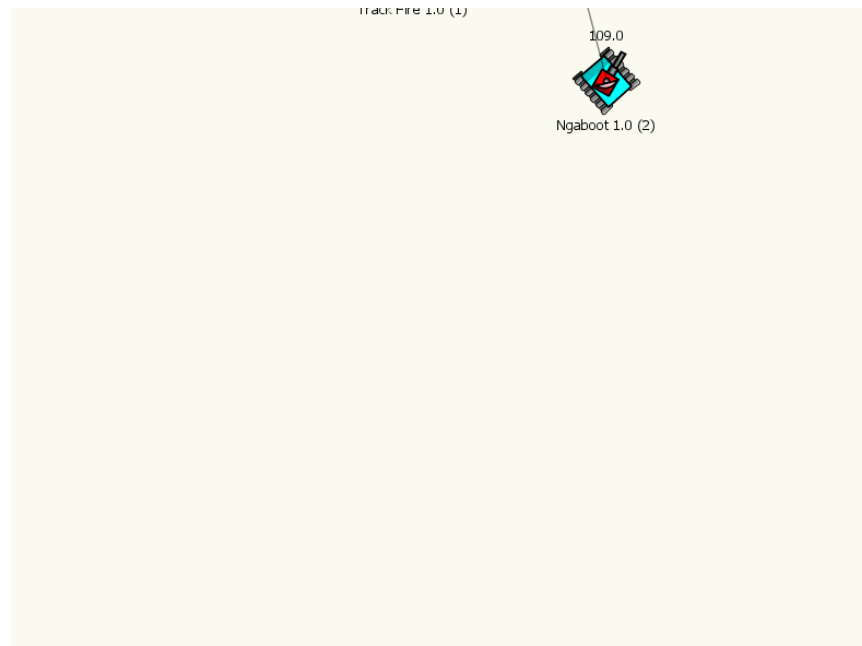
Gambar 3. Pengujian satu Ngaboot vs SurvivorBot

Tabel 3. Analisis Pengujian Tiga

Parameter	Bot Utama	Alternatif Bot
Skor Akhir	1254	50
Energi Tersisa	350	0
Jumlah Tembakan	624	36
Akurasi Tembakan	112	0
Hasil Pertandingan	8/0/0	0/8/0

4.3.4 Pengujian 4:Ngaboot (Main Bot) VS TrackFireBot (Bot Asisten)

Pengujian ini dilakukan dengan mempertemukan bot Ngaboot dengan TrackFireBot milik Asisten Dosen. Pengujian ini mempertemukan dua bot agresive. Ngaboot dan TrackFireBot sama sama memberikan tekanan dari serangan yang sangat kuat dan aktif



Gambar 4. Pengujian satu Ngaboot vs TrackFireBot

Tabel 4. Analisis Pengujian Empat

Parameter	Bot Utama	Bot Asisten
Skor Akhir	1667	821
Energi Tersisa	350	150
Jumlah Tembakan	1046	572
Akurasi Tembakan	131	69
Hasil Pertandingan	7/3/0	3/7/0

4.4 Analisis Hasil Pengujian

Berdasarkan seluruh hasil pengujian, Ngaboot menunjukkan performa paling unggul dibanding bot lainnya baik dalam duel satu lawan satu maupun pertarungan multi-bot. Pada pengujian melawan RammerBot, Ngaboot berhasil memenangkan seluruh ronde karena mampu menjaga jarak dan menghasilkan bullet damage yang tinggi, sedangkan RammerBot terlalu bergantung pada strategi tabrakan sehingga mudah diserang saat mendekat. Pada pengujian melawan SniperBot, Ngaboot kembali mendominasi karena memiliki mobilitas dan tempo serangan yang lebih agresif dibanding strategi predictive shooting milik SniperBot yang cenderung lebih lambat. Saat melawan SurvivorBot, Ngaboot juga unggul karena tekanan serangannya mampu mengatasi strategi evasive movement yang dimiliki SurvivorBot. Dalam pengujian seluruh bot di satu arena, Ngaboot tetap memperoleh skor tertinggi sehingga menunjukkan bahwa strategi hybrid agresif adaptif yang digunakan memiliki keseimbangan terbaik antara serangan, mobilitas, dan kemampuan bertahan. Sementara itu, RammerBot unggul dalam ram damage namun lemah dalam pertahanan, SniperBot memiliki akurasi yang cukup baik tetapi kurang efektif dalam tempo permainan cepat, dan SurvivorBot memiliki kemampuan bertahan hidup yang baik namun kurang agresif dalam menghasilkan damage maupun kemenangan. Pada pengujian 4 menggunakan Bot milik Asisten bernama TrackFire yang diuji dengan Ngaboot dari hasil pengujian tersebut bahwa Ngaboot masih lebih unggul dibanding TrackFire meskipun dalam segi performa tidak terlalu berbeda jauh.

BAB V: KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil desain, implementasi, dan pengujian turnamen, dapat disimpulkan bahwa:

- Algoritma Greedy sangat efektif dan andal diaplikasikan pada permainan waktu nyata seperti Robocode Tank Royale karena bebas dari beban komputasi berat, sehingga menghasilkan kecepatan respons gerak yang instan.
- Kelemahan inheren dari algoritma Greedy terletak pada sifat pandang pendeknya yang mengabaikan parameter global penting (seperti sisa darah musuh jarak menengah), sehingga pada skenario pertempuran massal yang kompleks, keputusan yang diambil sering kali bersifat suboptimal secara taktis.

5.2 Saran

Untuk mengatasi keterbatasan sifat dasar algoritma Greedy, penelitian pengembangan robot kecerdasan buatan selanjutnya disarankan untuk mengombinasikan logika Greedy dengan metode metaheuristik atau prediktif, seperti:

- Penerapan *Linear/Circular Target Leading* menggunakan kalkulasi trigonometri dinamis untuk memprediksi posisi koordinat masa depan musuh agar akurasi tembakan jarak jauh meningkat tajam.
- Implementasi *Machine Learning* atau Jaringan Saraf Tiruan (ANN) yang digabungkan dengan algoritma genetika agar nilai ambang batas transisi status strategi dapat berevolusi secara otomatis beradaptasi dengan taktik bertarung musuh yang unik.

LAMPIRAN

LAMPIRAN A : Tautan Repository Github

<https://github.com/davidelbai/Tugas-Besar-Strategi-Algoritma.git>

LAMPIRAN B ; Tautan Video Demo

<https://youtu.be/TubesStimaRC>

LAMPIRAN C : Tabel Pemenuhan Tugas

No	Poin	Ya	Tidak
1	Bot dapat dijalankan pada Engine yang sudah dimodifikasi asisten.		
2	Membuat 4 solusi greedy dengan heuristic yang berbeda.		
3	Membuat laporan sesuai dengan spesifikasi.		
4	Membuat video bonus dan diunggah pada Youtube.		

DAFTAR PUSTAKA

- Prasad, S. (n.d.). Classical and Heuristic Approaches for Mobile Robot Path Planning: A Survey. *Robotics*, 12(4), 93. 10.3390/robotics12040093
- Sabara, E. (2019). Penerapan Algoritma Greedy Dalam Pencarian Jalur Berdasarkan Bobot Terkecil. *Jurnal Media Infotama*, 15(2), 105-110. 1858-2680
- Yeşil, Ç. (n.d.). A Novel Behavioral Strategy for RoboCode Platform Based on Deep Q-Learning. *IEEE Access*, 10(1), 105869-105878.